Lập trình mạng · Chương 2

# TCP: timeout, half-close và lỗi kết nối

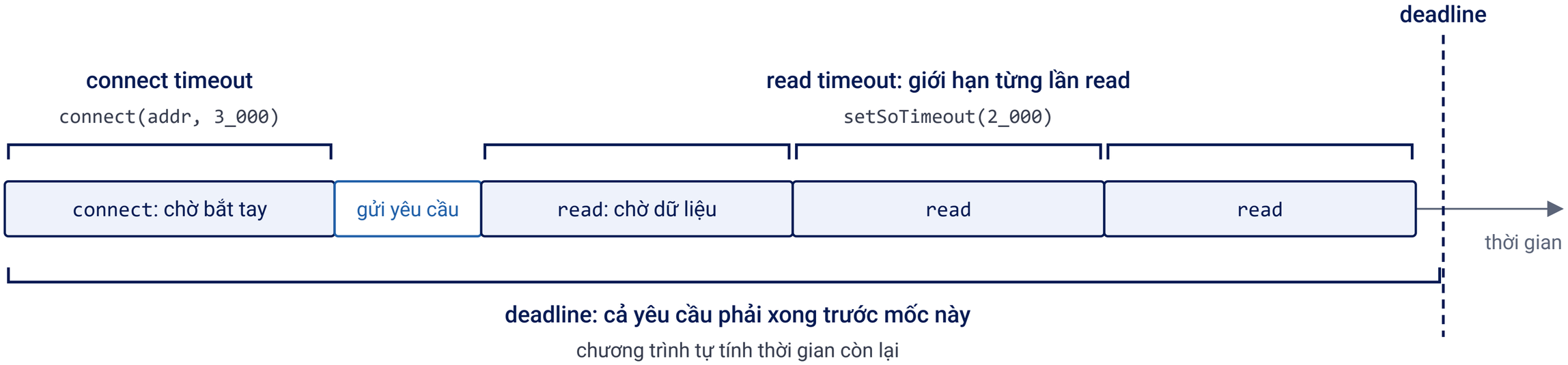
TS. Đặng Ngọc Hùng

# Không đặt giới hạn thì connect và read có thể chờ rất lâu

1. `connect()` chờ đến khi hệ điều hành thôi gửi lại SYN: **21 giây** trên Windows 11, khoảng **2 phút** trên Linux; `readLine()` chờ **không giới hạn**
2. Khi kết nối hỏng, chương trình cần biết **hỏng ở giai đoạn nào** để báo lỗi đúng
3. Gửi xong mà vẫn chờ phản hồi: client cần **báo hết dữ liệu**, để server không phải chờ đến hết thời gian

> **Câu hỏi của phần:** giới hạn thời gian chờ thế nào, phân biệt các lỗi kết nối ra sao, và báo hết dữ liệu bằng cách nào?

# Timeout giới hạn từng thao tác chờ, deadline giới hạn cả yêu cầu




**Connect timeout** và **read timeout** giới hạn một thao tác; hết hạn thì ném `SocketTimeoutException`. **Deadline** giới hạn cả yêu cầu.

# Lỗi kết nối TCP phân biệt được theo giai đoạn và kiểu ngoại lệ

### Khi kết nối: lời gọi `connect`

**Máy đích trả RST**

`ConnectException: Connection refused`

gần như ngay lập tức (22 ms trên loopback)

**Gói SYN không có phản hồi**

`SocketTimeoutException: Connect timed out`

khi hết connect timeout

### Khi trao đổi dữ liệu: lời gọi `read`

**Không có dữ liệu trong read timeout**

`SocketTimeoutException: Read timed out`

kết nối vẫn còn, socket vẫn dùng được

**Đầu kia hủy kết nối bằng RST**

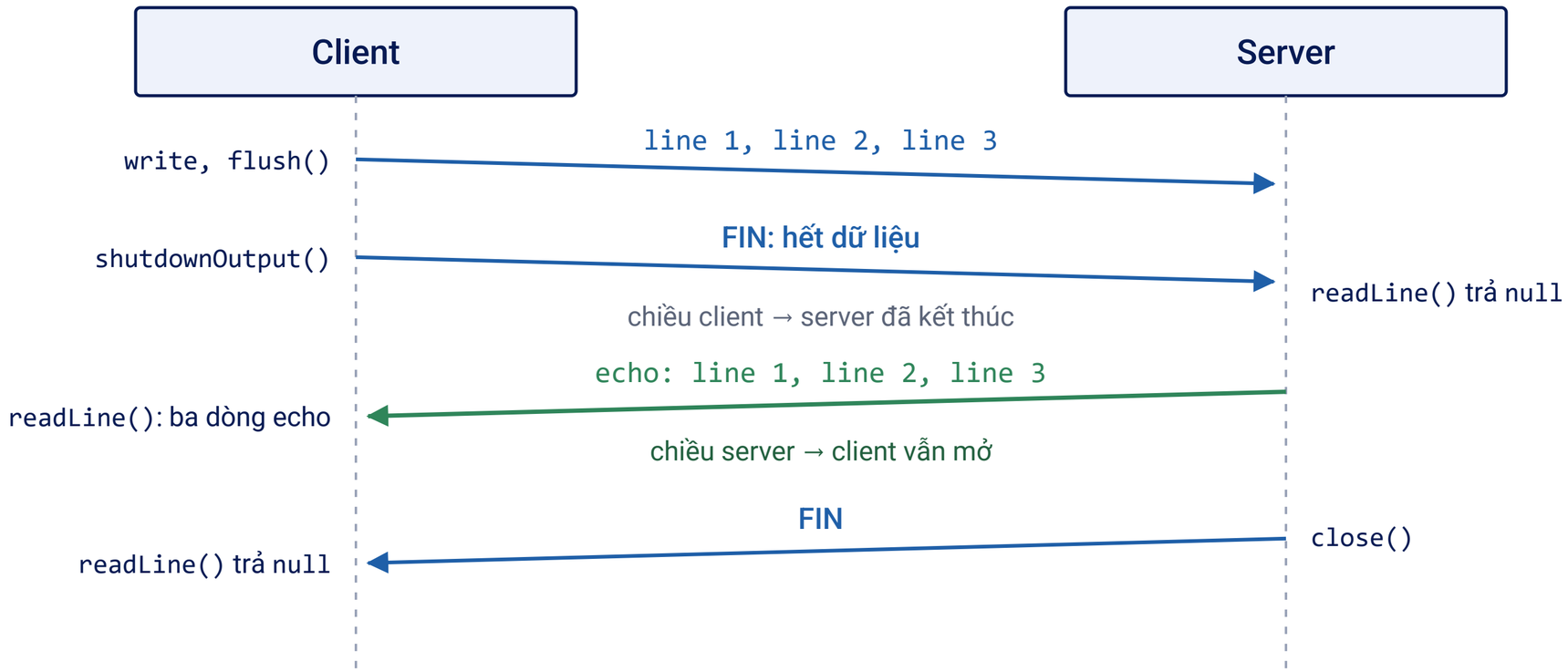
`SocketException: Connection reset`

ví dụ: đầu kia đóng socket khi còn dữ liệu chưa đọc

`read` trả `−1`, `readLine` trả `null`: đầu kia đã kết thúc chiều gửi. **Không phải lỗi.**

> Kiểu ngoại lệ và thời điểm xảy ra cho biết lỗi ở **giai đoạn kết nối** hay **khi trao đổi dữ liệu**.

# Half-close kết thúc chiều gửi, chiều nhận vẫn mở




**Half-close**: `shutdownOutput()` gửi FIN, đầu kia đọc được −1; chiều ngược lại vẫn trao đổi được.

# Read timeout hết hạn sau 2 giây, connect timeout hết hạn sau 3 giây

Java

```
long start = System.nanoTime();
try (Socket socket = new Socket()) {
    socket.connect(new InetSocketAddress(host, 9199), 3_000);  // connect timeout: 3 giây
    socket.setSoTimeout(2_000);  // read timeout: mỗi lần read chờ tối đa 2 giây
    socket.getOutputStream().write((message + "\n").getBytes(UTF_8));  // gửi một dòng
    var in = new BufferedReader(new InputStreamReader(socket.getInputStream(), UTF_8));
    String reply = in.readLine();  // chờ dòng được gửi lại
    System.out.println(host + ": reply \"" + reply + "\" after " + millis(start) + " ms");
} catch (IOException e) {  // loại lỗi: tên lớp ngoại lệ và thông báo
    System.out.println(host + ": " + e.getClass().getSimpleName() + ": " + e.getMessage()
            + " after " + millis(start) + " ms");
}
```

Kết quả chạy

```
127.0.0.1: SocketTimeoutException: Read timed out after 2045 ms
192.0.2.1: SocketTimeoutException: Connect timed out after 3014 ms
```

# Sau shutdownOutput(), client vẫn đọc được phản hồi

Java

```
try (Socket socket = new Socket("127.0.0.1", 9199)) {
    var out = new BufferedWriter(new OutputStreamWriter(socket.getOutputStream(), UTF_8));
    var in = new BufferedReader(new InputStreamReader(socket.getInputStream(), UTF_8));
    out.write("line 1\nline 2\nline 3\n");
    out.flush();  // đẩy dữ liệu khỏi buffer của ứng dụng trước
    socket.shutdownOutput();  // half-close: hết dữ liệu gửi, chiều nhận vẫn mở
    String line;
    while ((line = in.readLine()) != null) {  // đọc mọi dòng được gửi lại
        System.out.println("echoed: " + line);
    }
}
```

Kết quả chạy

```
sent 3 lines, output shut down
echoed: line 1
echoed: line 2
echoed: line 3
readLine() returned null: server closed the connection
```

- `flush()` trước, `shutdownOutput()` sau: `shutdownOutput()` **không** flush buffer của ứng dụng

# Read timeout không giới hạn cả yêu cầu: đầu kia gửi chậm vẫn giữ được kết nối

```java
socket.setSoTimeout(2_000);  // read timeout: mỗi lần read chờ tối đa 2 giây
String reply = in.readLine();  // chờ dòng được gửi lại
```

Kết quả chạy

```
127.0.0.1: reply "hello" after 5069 ms
```

- Server gửi mỗi giây một byte: mỗi lần read chờ chưa tới 2 giây, read timeout **không lần nào** hết hạn
- Cần **deadline** cho cả yêu cầu: trước mỗi lần read, đặt read timeout bằng **thời gian còn lại**

# Câu hỏi 1

## Client gọi socket.setSoTimeout(2000) ngay sau khi kết nối. Lời gọi này giới hạn điều gì?

A. Thời gian của cả yêu cầu, từ lúc kết nối đến khi nhận đủ phản hồi.

B. Thời gian mỗi lần read chờ dữ liệu; quá 2 giây chưa có byte nào thì read ném SocketTimeoutException.

C. Thời gian lời gọi connect chờ bắt tay.

D. Thời gian kết nối được phép tồn tại trước khi tự đóng.

# Câu hỏi 1

## Client gọi socket.setSoTimeout(2000) ngay sau khi kết nối. Lời gọi này giới hạn điều gì?

A. Thời gian của cả yêu cầu, từ lúc kết nối đến khi nhận đủ phản hồi.

**B. Thời gian mỗi lần read chờ dữ liệu; quá 2 giây chưa có byte nào thì read ném SocketTimeoutException.**

C. Thời gian lời gọi connect chờ bắt tay.

D. Thời gian kết nối được phép tồn tại trước khi tự đóng.

setSoTimeout đặt read timeout, áp cho từng lần read. Connect timeout là tham số của `connect()`; giới hạn cả yêu cầu là deadline, chương trình tự tính. Kết nối không tự đóng khi hết read timeout.

# Câu hỏi 2

**Client gọi `shutdownOutput()` sau khi gửi xong yêu cầu. Sau lời gọi này, điều nào đúng?**

A. Kết nối đóng hẳn; client không đọc được phản hồi nữa.

B. Dữ liệu còn trong BufferedWriter được tự động flush trước khi chiều gửi kết thúc.

C. Client vẫn đọc được phản hồi; server đọc được −1 sau khi nhận hết dữ liệu.

D. Server nhận SocketException: Connection reset.

# Câu hỏi 2

## Client gọi `shutdownOutput()` sau khi gửi xong yêu cầu. Sau lời gọi này, điều nào đúng?

A. Kết nối đóng hẳn; client không đọc được phản hồi nữa.

B. Dữ liệu còn trong BufferedWriter được tự động flush trước khi chiều gửi kết thúc.

**C. Client vẫn đọc được phản hồi; server đọc được −1 sau khi nhận hết dữ liệu.**

D. Server nhận SocketException: Connection reset.

`shutdownOutput()` chỉ kết thúc chiều gửi: TCP gửi FIN, server đọc được −1, chiều từ server về client vẫn mở. Dữ liệu còn trong buffer của ứng dụng không được gửi; phải gọi `flush()` trước. FIN không phải RST nên không có connection reset.

# Tổng kết

1. **Connect timeout** giới hạn bắt tay; **read timeout** (`setSoTimeout`) giới hạn từng lần read; **deadline** giới hạn cả yêu cầu

2. Lỗi theo giai đoạn: refused, connect timed out khi kết nối; read timed out, **connection reset** khi trao đổi dữ liệu

3. **Half-close** (`shutdownOutput`): đầu kia đọc được −1, chiều nhận vẫn mở

**Từ khóa cần nhớ** connect timeout · read timeout · SocketTimeoutException · deadline · half-close · shutdownOutput · connection reset

# Mở rộng

### Chủ đề nâng cao

- TCP keepalive và giới hạn của nó
- Trạng thái TIME_WAIT sau khi đóng kết nối

### Bài tập mở rộng

Thêm deadline 5 giây cho mỗi lần gọi của echo client: hết hạn thì đóng kết nối và báo lỗi theo giai đoạn (kết nối, gửi, chờ phản hồi).

Tiếp theo

# Bài toán ứng dụng: web server tối giản trên TCP